

# PRISM — Target Architecture (mapped to your actual code)

## What I found in your zip (important — read this first)

- **Current stack confirmed:** Vercel + Express (`api/index.ts`), Firebase (auth + Firestore), Gemini (not Claude) via `@google/genai`, React 19 + Vite frontend, D3 for charts.
  - **You already have a real client-side ML engine** — `biasEngine.ts` + `bias.worker.ts` run **MobileBERT zero-shot classification** and **MiniLM embeddings** entirely in-browser via `@huggingface/transformers` (ONNX, WebGPU/WASM). This is huge: it means the "classical/local ML" tier we discussed isn't hypothetical — it's already built and already free (runs on the user's device, costs you nothing).
  - **Confirmed fabrication issue (from earlier warning):** `api/index.ts` has a `generateSandboxGNewsArticles()` function that invents fake Reuters/AP/WSJ headlines when no `GNEWS_API_KEY` is set, and `App.tsx` hardcodes `INITIAL_ARTICLES`/`INITIAL_SOURCES` with fabricated stats ("cryptographic provenance tracing," invented credibility scores). This is your Phase 0 — it must be replaced with real data before anything else matters.
  - **In-memory vote store** (`votesStore: Vote[]` in `api/index.ts`) — resets on every server restart/redeploy. No persistence.
  - **No caching layer** — every `/api/gemini/analyze` call re-runs the full Gemini pipeline, even for a repeat query.
  - `requireAuth` is already solid — Firebase ID token verification is correctly implemented. Keep this.
-

## Target architecture

### INGESTION (new)

- Scheduled job replaces on-demand `fetchGNewsArticles()`
- Real RSS feeds (Reuters, AP, BBC, etc.) + GNews API
- Runs every 15-30 min via Vercel Cron or a small worker



### DEDUP + CACHE (new – Upstash Redis, free tier)

- Same story from 5 outlets = 1 entry
- Cached Gemini/Claude verdicts keyed by story hash



### CLASSICAL ML LAYER (mostly EXISTS – extend it)

- `biasEngine.ts` + `bias.worker.ts` already do zero-shot bias/topic classification + embeddings, client-side, free
- NEW: move source-credibility scoring here too (outlet reputation + cross-outlet corroboration count), not hardcoded numbers



### VERIFY TRIGGER (modify existing `/api/gemini/analyze`)

- Default: every article gets classical-ML score + source link
- Full multi-agent debate only on: user click, OR top trending, OR classical-ML flags high uncertainty



### TIERED LLM ROUTER (new – wraps existing Gemini calls)

- Free tier: Groq/Gemini free quota → default debate engine
- Pro tier: Claude API → paying users / flagged high-stakes stories



### STORAGE (replace hardcoded arrays + in-memory `votesStore`)

- Firestore (you already have `firebase-admin`) OR migrate to Supabase Postgres if you want relational queries for the map view



Frontend (`TransparencyFeed.tsx`, `BiasSimilarityCanvas.tsx`, etc.)

# File-by-file change list

`api/index.ts` (the biggest changes live here)

| Current                                                                                                    | Change to                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>generateSandboxGNewsArticles()</code><br>fabricates fake headlines                                   | <b>Delete this.</b> If no API key, show an honest "no live data configured" state — never fabricate sourced content                                                                                                          |
| <code>fetchGNewsArticles()</code> runs on-demand per request, 10-min in-memory cache                       | Move to a <b>scheduled ingestion job</b> (Vercel Cron hitting a <code>/api/cron/ingest</code> route every 15-30 min), write results to real storage, not a request-scoped cache                                              |
| <code>votesStore: Vote[]</code> in-memory array                                                            | Move to Firestore ( <code>db.collection('votes')</code> ) — you already have <code>firebase-admin</code> initialized, this is a small change                                                                                 |
| <code>/api/gemini/analyze</code> always runs full Gemini call                                              | Split into two paths: <b>cheap path</b> (classical ML score, near-instant, free) as default response; <b>expensive path</b> (current Gemini multi-agent logic) only fires on explicit <code>?deep=true</code> verify request |
| Candidate model fallback chain ( <code>gemini-3.5-flash</code> → <code>gemini-3.1-flash-lite</code> → ...) | Keep this pattern — it's actually good defensive design. Add a <b>Claude candidate</b> at the top of the chain for Pro-tier requests specifically                                                                            |
| No caching before calling Gemini                                                                           | Add Upstash lookup by <code>hash(searchQuery + category)</code> before every Gemini/Claude call — return cached result if present                                                                                            |

`src/App.tsx`

| Current                                                                                        | Change to                                                                                                                                             |
|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>INITIAL_SOURCES</code> and <code>INITIAL_ARTICLES</code> hardcoded with fabricated stats | Replace with a fetch from real storage on mount. If empty (no data yet), show a genuine empty/loading state — not fake placeholder "verified" content |
| Fetch proxy rewrite ( <code>window.fetch</code> override for <code>/api/</code> )              | Fine as-is, no change needed                                                                                                                          |

`src/services/biasEngine.ts` + `src/workers/bias.worker.ts`

### Current

### Change to

Zero-shot classifier + embeddings, working well

**Keep exactly as-is** — this is your free, always-on, client-side scoring layer. It's the foundation of the "classical ML" tier we discussed, no rebuild needed

Only used for `BiasSimilarityCanvas` (the similarity graph feature)

**Expand its role:** also call `biasEngine.classify()` on every ingested article for a first-pass bias label, before deciding if it needs full LLM debate

`src/types.ts`

### Current

### Change to

`Article.sourceCredibility: number` — implies a single hand-set number

Add fields to distinguish tiers:  
`verificationTier: "basic"

## New files needed

- `api/cron/ingest.ts` — scheduled ingestion (replaces on-demand fetch pattern)
- `src/services/cache.ts` — Upstash client wrapper
- `src/services/llmRouter.ts` — the tiered router deciding Groq/Gemini vs Claude based on user tier + story priority
- `src/services/credibilityScorer.ts` — outlet-reputation + corroboration-based scoring (this is your classical ML credibility layer, separate from the existing bias classifier)

## Sequencing (do not reorder)

1. **Kill the fabricated data** — remove `generateSandboxGNewsArticles()` and hardcoded `INITIAL_ARTICLES`/`INITIAL_SOURCES`. Non-negotiable, do this before anything else.
2. **Real ingestion + Firestore persistence** for articles and votes.
3. **Wire `biasEngine.ts` into the ingestion pipeline** (it already exists — just call it on every new article instead of only on-demand for the canvas view).
4. **Add Upstash caching** in front of the Gemini calls.
5. **Split `/api/gemini/analyze` into cheap/expensive paths.**
6. **Add the Claude candidate + tiered router** once the cheap path is stable.

*Ready to start writing actual code for step 1 (removing the fabricated data and wiring real ingestion) whenever you want to move into implementation.*

